Agent = Harness(Model):
The Five Primitives of Every Agent Harness

Chase Dovey
Average Joes Lab
https://cdovey.dev

Abstract

Large language model (LLM) agents are usually described by the model inside them, yet the same model can behave as a careful engineer in one system and fail to finish a task in another. What differs is everything around the model: the software that decides what the model sees, what it can do, and when it runs again. Practitioners call this a *scaffold*, a *framework*, a *runtime* or, increasingly, a *harness*, but the term has no shared definition, and published comparisons rarely separate the harness from the model. We take the position that every LLM agent is a model wrapped in a harness, *Agent* = *Harness*(*Model*), and that every harness, however large, is built from the same five primitives: Input, which captures what comes in; Context, which assembles what the model is shown; Model interface, which requests a response from the model; Output, which carries the response out; and Control flow, which decides whether to go again, hand back, or stop. We argue that each primitive is necessary, by showing the specific failure of an agent built without it, and that together they are sufficient to express the agents in common use. We then suggest how the concerns of production use (persistence, safety and security, observability, resilience, performance and evaluation) fold into these primitives rather than standing beside them. In a case study, we map widely used harnesses, from command-line coding agents to research systems and agent frameworks, onto the five primitives, and argue that the differences between them are implementation decisions within each primitive, not differences in kind. We close with what this view implies for how agents are built, compared, and trained.

1 Introduction

A language model does one thing: given a sequence of tokens, it produces a distribution over the next token. It cannot see a file it was not shown, remember a conversation it was not sent again, or carry out a command it wrote. Yet language models are now routinely described as agents that read codebases, run tests, browse the web and fix bugs. The gap between what a model is and what an agent does is filled by software around the model. That software is the subject of this paper.

The software goes by many names. Evaluations call it a *scaffold* or *scaffolding* [1, 11]; libraries call themselves *frameworks* or *SDKs*, or an *orchestration framework and runtime* [10]; and the term *harness* has become common among practitioners who build coding agents [30]. The names point at the same thing, but none comes with a definition precise enough to say what the thing contains, where it ends and the model begins, or how two instances of it differ. As a result, agents are compared by the model they use, and the harness is treated as an unremarkable detail, even though results on the same benchmark with the same model can differ widely depending on the system around it (§7).

Position. We argue three claims.

1. **Every LLM agent is a model wrapped in a harness**: *Agent* = *Harness*(*Model*). The model maps tokens to a next token and nothing more. The harness is everything else.
2. **Every harness is built from the same five primitives**: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. Each is necessary: remove one and the agent fails in a specific, predictable way. Together they are sufficient to express the agents in common use.
3. **The differences between harnesses are implementation decisions within the primitives**, not differences in kind. Whether an agent has one tool or forty, a graph or a loop, a repository map or a summarizer, it answers the same five questions differently.

Why it matters. A shared vocabulary turns a vague object into one that can be designed, compared and studied. If the harness is named and decomposed, benchmark results can report which harness produced them and isolate its effect; a failure can be traced to the primitive responsible; a harness can be ported to a new model by changing one primitive; and a model can be trained for the harness it will run in. We return to each of these in §7.

Contributions.

- A definition of the harness, and of the boundary between harness and model (§3).
- Five primitives, each with its role in the agent loop, an argument for its necessity, and the design space within it (§4).
- A suggestion of how six production concerns fold into the primitives (§5).
- A case study mapping widely used harnesses onto the primitives, and the implementation decisions that distinguish them (§6).
- Implications for building, comparing and training agents, and a discussion of alternative views (§7, §8).

Our claims are drawn from building a harness from nothing, one primitive at a time, around a fixed model and a fixed task, and observing what each version could and could not do; the resulting reference harness, *quark*, is about fifty lines and is reproduced in Appendix A. The code, transcripts and evaluation are public.[1]

2 Background

Agents. In artificial intelligence and cognitive science, an agent is a system that perceives its environment and acts upon it [16], usually in a repeating cycle of perception, decision and action, supported by memory. Cognitive science distinguishes a short-lived *working memory* from long-term memories that are *episodic* (what happened), *semantic* (what is true) and *procedural* (how to do things).

Language agents. The loop that harnesses implement goes back to ReAct [29], which interleaves a model's reasoning with actions whose results are returned to it as observations. Later systems elaborated the loop with patterns that, in our terms, live in particular primitives. Reflexion [20] has the agent write reflections on failed attempts and carries them into the next attempt: a decision in CONTROL FLOW, which runs the attempts, and in CONTEXT, which carries the reflections forward. MemGPT [15] manages what is kept inside and outside a limited context window, paging information in and out as needed: a decision in CONTEXT.

[1] https://github.com/averagejoeslab/building-agents

CoALA. Sumers et al. [22] proposed *Cognitive Architectures for Language Agents*, which organizes language agents by their memory (working, episodic, semantic and procedural), their action space (external *grounding* actions and internal *retrieval*, *reasoning* and *learning* actions), and a decision cycle of planning and execution. CoALA describes *what an agent is*, as a cognitive architecture. Our concern is narrower and more operational: *what software an agent is made of*, as a system an engineer builds. The two views are complementary, and §4 maps between them.

Agent-computer interfaces. Yang et al. [28] showed that the interface through which a model acts on a computer (the commands it is given, their output format, and guardrails such as linting edits) substantially changes performance on software engineering tasks, and named this the *agent-computer interface*. In our terms, their interface is a set of decisions within the OUTPUT and CONTEXT primitives.

Practitioner guidance. Schluntz and Zhang [18] distinguishes *workflows*, in which code fixes the sequence of model calls, from *agents*, in which the model directs its own process and tool use, and describes composable patterns such as prompt chaining, routing and orchestrator–worker designs. We treat both workflows and agents as harnesses that differ in their CONTROL FLOW (§6).

3 What a Harness Is

The model. We take a language model to be a function from a token sequence to a distribution over the next token, applied autoregressively until a stop token is produced:

$$TokensOut = Model(TokensIn).$$

Chat templates, tool-call formats and reasoning tokens are conventions for laying out *TokensIn* and reading *TokensOut*; models can learn to emit calls to external tools [17], and they learn these conventions in training, but the calls are carried out, and the conventions written and parsed, by something else.

The harness. We define the *harness* as everything in an agent that is not the model: the software that turns the world into tokens for the model, and the model's tokens into effects on the world. An agent is a model inside a harness:

$$Agent = Harness(Model).$$

The definition draws the boundary at the model's weights and its sampling. Everything the model is shown, everything it is able to do, and everything that happens between one call and the next is on the harness side. This includes things often attributed to "the model", such as memory across sessions (the model has none; the harness stores and retrieves it), tool use (the model emits a request; the harness carries it out), and knowledge of the current date or working directory (the model knows only what the harness writes into its context).

The loop. Figure 1 shows the shape every harness shares. Something comes in: a request from a person, an event, or the result of an earlier action. The harness assembles a context around it and sends that context to the model. The model runs its own inner loop, one token at a time, until it stops. The harness handles the response: showing words to a person, or carrying out the actions it requests. If an action produced a result, the result comes back in, and the loop runs again; if not, control returns to the person, or the agent stops.

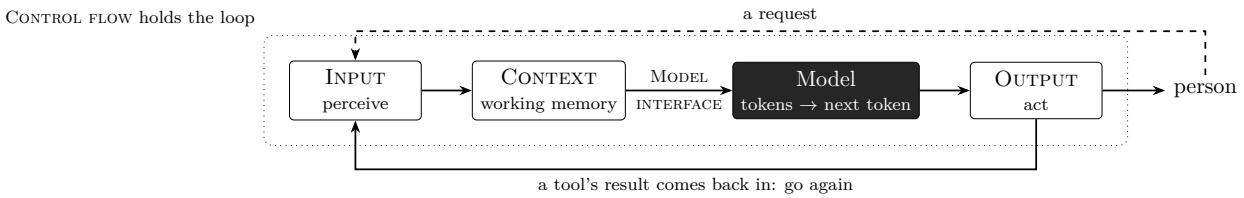


Figure 1: The agent loop. The harness's five primitives surround the model; Control flow decides whether the loop runs again, hands back, or stops.

Algorithm 1 The five primitives of a harness. Lines are labelled with the primitive they implement.

```
 1: procedure ControlFlow( )
 2:     conversation ← [ ]
 3:     loop                                          ▷ outer loop: one request at a time
 4:         conversation.append(User(Input( )))        ▷ Input: from a person
 5:         loop                                      ▷ inner loop: one request, carried through
 6:             ctx ← Context(conversation)           ▷ Context
 7:             response ← ModelInterface(ctx)        ▷ Model interface
 8:             conversation.append(Assistant(response))
 9:             results ← Output(response)            ▷ Output: show words, run actions
10:             if results = ∅ then break             ▷ nothing ran: hand back
11:             conversation.append(User(results))    ▷ Input: from the world
```

Not every harness is an agent's. A program that makes one model call and prints the reply has a harness, but a trivial one: its input is fixed, its context is one message, its output is printed, and its control flow is a straight line. The five primitives describe this case as well; what turns a harness into an *agent's* harness is a Control flow that feeds the results of actions back to the model (§4).

4 The Five Primitives

We identify five primitives. For each we give its role, the evidence that it is necessary (the failure of a harness built without it), and the design space within it, which is where harnesses differ. The necessity arguments come from building a harness outward from the model, adding one primitive at a time, and running each version on the same task: a small Python project with a failing test, and the request *"The tests are failing. Find out why and fix it."* The transcripts are in the repository.

Algorithm 1 gives the whole harness as pseudocode. Every harness we examine in §6 is an elaboration of it.

4.1 Model interface: reach the model

Role. Send an assembled context to the model and return its response in a form the rest of the harness can read: words, and structured requests to act. **Without it.** There is no agent at all, only weights. **With only it.** The model answers, but cannot act; asked to fix failing tests, it explains that it cannot see the code. A request written into the program means nobody else can ask it anything. **Design space.** Hosted API or local inference; one provider or many (directly or through a translation layer); native tool-call formats or tool calls parsed from text; streaming or whole responses; and the translation between the harness's internal message format and the model's

chat template. The model interface is the only primitive that knows which model is inside, which is why changing models means changing it alone.

4.2 Input: capture what comes in

Role. Capture what enters the agent: requests from a person, and results from the world. **Without it.** The agent can only ever do one, fixed thing. **Design space.** The *source* of input (a terminal prompt, an editor panel, a chat channel, a command-line argument for non-interactive use, an issue tracker, a scheduled trigger or a webhook) and its *kind* (text, images, files, the results of actions, and interrupts). The second kind of input, results from the world, is easy to overlook and is what makes acting worthwhile: an action's result is new perception.

4.3 Output: carry the response out

Role. Handle what the model returns: show its words, and carry out its requests to act, collecting each result. **Without it.** The model can only describe what it would do; nothing it writes is carried out, and asked to fix the tests it asks to be shown the code. **With it but without control flow.** The agent acts once and stops. It lists the project's files, and the list goes nowhere: the result of the action never reaches the model. **Design space.** The *tool set*, from a single general tool (a shell) to dozens of specific ones (read, write, edit, search, browse, a computer-use interface); the *action format*, from structured tool calls to code the model writes and the harness executes [24] to edit formats such as search-and-replace blocks or diffs; the *execution environment*, from the host machine to a container or remote sandbox; and the *presentation* of results, including truncation, formatting and error messages, which Yang et al. [28] show matter to performance. The model only ever requests an action; Output decides whether and how it happens, which is why the guardrails of §5 live here.

4.4 Control flow: decide what happens next

Role. Shape the loop: send results back and go again, hand back to the person, or stop. **Without it.** The agent acts once. With it, the same model, tool and request become an agent: it lists the files, reads them, runs the tests, reads the failure, edits the code, runs the tests again and reports, then hands back. Because the whole conversation returns on every call, a follow-up such as *"Commit the fix"* needs no explanation. **Design space.** A single open loop that runs until the model stops requesting actions; a fixed pipeline of model calls (a *workflow*); an explicit graph of steps; delegation to sub-agents with their own contexts; planning or approval modes that change what is allowed; and stopping rules such as step limits, budgets and interrupts. Schluntz and Zhang [18]'s workflows and agents differ here, and only here.

4.5 Context: assemble what the model sees

Role. Assemble, for each call, everything the model is shown: instructions, the tools it can use, the conversation so far, and anything retrieved from memory. Context is the agent's working memory, rebuilt on every call. **Without it.** The agent works, but does not know what it is, where it is, or what day it is. Asked who it is and where it is working, it must spend actions finding out: in our runs it ran `pwd`, `whoami` and `uname` before answering, and still described itself only as "an AI assistant". A model with no such context can also guess wrongly about what it is. One sentence of context (a name, the tool it acts through, the working directory and the date) answers the question without any action. **Design space.** What goes into the *instructions* (identity, rules, the agent's own

Table 1: The five primitives, the agent cycle they serve, and the corresponding CoALA components.

Primitive	Role	In the agent cycle	CoALA
Input	Capture requests and results	Perceive	Observations
Context	Assemble what the model sees	Working and long-term memory	Working memory; retrieval
Model interface	Request a response	Decide	Reasoning (by the LLM)
Output	Show words, carry out actions	Act	Grounding actions; learning (writes)
Control flow	Go again, hand back, or stop	The cycle	Decision cycle

capabilities, project-specific guidance from files such as `AGENTS.md`); how the *tools* are described; how *long-term memory* is brought in (preloaded, retrieved on demand, or searched by the agent through its own tools); and how the context is kept within the window when it grows, by dropping old turns, summarizing them, or including only what the moment needs. Context also carries roles, and roles carry authority: instructions in the system role outrank text that arrives in a tool result, a distinction that matters for security (§5).

4.6 Necessity and sufficiency

Each primitive answers a question no other primitive answers: *how is the model reached* (Model interface), *what comes in* (Input), *what is done with what comes out* (Output), *what happens next* (Control flow) and *what the model is shown* (Context). Removing any one produces the specific failure described above, so none is redundant. For sufficiency, we offer the case study of §6: every capability we found in widely used harnesses is an implementation of one of the five, or a production concern that folds into them. We discuss candidates for a sixth primitive in §8.

4.7 Relation to CoALA

The primitives give an operational reading of CoALA's components [22]. CoALA's working memory is Context; its long-term memories are stores that Context reads from and Output writes to; its grounding actions are carried out by Output and their observations return through Input; its internal reasoning happens in the model, reached through Model interface; and its decision cycle is Control flow. Table 1 summarizes the mapping.

5 Production Concerns Fold into the Primitives

A harness that works is not yet a harness one would leave running. Production systems add persistence, guardrails, logging, retries, caching and evaluation, and it is tempting to treat these as further components. We suggest instead that each is a *concern*: a question asked of every primitive, whose answer changes how some primitives are implemented, rather than a new primitive. Table 2 is the matrix we found useful; we offer it as a suggestion, not a fixed taxonomy.

Persistence. The model remembers nothing between calls, so anything that should outlast a session is stored by the harness and brought back by Context: a log of every message (episodic memory), facts (semantic memory), and reusable procedures or project instructions (procedural

Table 2: Production concerns as questions asked of each primitive. Each cell names what the concern touches in that primitive; "—" means we found nothing essential there.

	Persistence *What should outlast the session?*	**Safety & security** *What could do harm, and who could make it?*	**Observability** *What should we see?*	**Resilience** *What could fail?*	**Performance** *What is slow or costly?*	**Evaluation** *What could a change break?*
INPUT	—	a way to interrupt	—	input ending	—	—
CONTEXT	memories; resumed sessions	untrusted content; poisoned memory	—	too long to send	resending everything	the instructions
MODEL INTERFACE	—	—	tokens, latency	a failing provider	waiting for the whole reply	the model
OUTPUT	—	harmful actions; secrets leaving	what each action did	a hanging action	large results	the tools
CONTROL FLOW	the task in progress	runaway loops; the audit record	each decision	a crash	—	the loop

memory). CONTROL FLOW can resume an interrupted session from the log. Checkpointing the *workspace*, so that an agent's changes can be rolled back, belongs here as well.

Safety and security. We separate two kinds of guardrail. *Safety* guards against the agent's own mistakes: a sandbox that limits where damage can happen, approvals before consequential actions, actions that are never allowed, step limits and interrupts. *Security* guards against someone turning the agent against its user: text in a file or web page that tries to steer the model (prompt injection), requests that would leak secrets, instructions planted in long-term memory, and tampering with the record of what was done. Both live mostly in OUTPUT, which carries out actions, and in CONTEXT, which decides how much authority each piece of text receives.

Observability. A record of each model call (tokens, latency, cost), each action and its result, and each refusal or stop, kept where the agent cannot alter it. It is written by CONTROL FLOW and OUTPUT, and read by people, not by the model.

Resilience. Retries and fallback providers in MODEL INTERFACE; time limits on actions in OUTPUT; summarization or truncation when a request is too long in CONTEXT; and resumption after a crash in CONTROL FLOW.

Performance. Caching the unchanged prefix of the context between calls; streaming responses; running independent actions in parallel; trimming large results; and using smaller models for auxiliary calls such as summarization.

Evaluation. Tasks with known answers, run in fresh environments, checked by what the agent did rather than what it said, and held out from any training. Evaluation is a concern of every primitive at once, because a change to any of them can change behaviour.

The concerns interact. Persistence creates a security surface (a poisoned memory steers every later session); streaming changes safety (an interrupt can now stop a reply mid-sentence); summarization changes persistence (the summary must say where the originals are kept). Treating

concerns as questions asked of each primitive makes these interactions visible, where treating them as separate components hides them.

6 Case Study: Existing Harnesses Through the Five Primitives

We now test the position against harnesses in wide use. If it holds, each should decompose into the five primitives with nothing left over, and the differences between them should be locatable as decisions within a primitive.

Method. We examined thirteen harnesses in three groups: command-line coding agents from model developers and the open-source community (Claude Code with its Agent SDK [2, 19], Codex CLI [13], Gemini CLI [5], Aider [4] and Cline [3]); research systems for software engineering (SWE-agent [28], mini-SWE-agent [23], OpenHands [25, 26] and Agentless [27]); and general-purpose agent frameworks (LangGraph [10], the OpenAI Agents SDK [14] and smolagents [6], with the original AutoGPT [21] for historical comparison). For each, we read the primary sources (official documentation, source code and papers) and recorded how it implements each primitive and each production concern. Products in this space change quickly; the mapping describes them as documented in October 2026, and source-code observations refer to Codex CLI at commit `2f05c0a` and Gemini CLI at commit `42c5182`. Where we could not confirm a detail in a primary source, we omit it.

6.1 Mapping

Tables 3, 4 and 5 give the mapping. Every capability we found has a home in one primitive or in a production concern; we found no capability that required a sixth primitive.

6.2 Frameworks expose the primitives

Agent frameworks differ from complete harnesses in *which* primitives they implement and which they leave to the developer, and so they show the primitives directly. LangGraph describes itself as focused entirely on orchestration: it provides Control flow as an explicit graph, and Context as typed state merged by reducers, while model calls and tool execution happen in user-written nodes [10]. The OpenAI Agents SDK provides a fixed Control flow, a runner loop that calls the model and then either finishes, hands off to another agent, or runs tool calls and loops, and draws an explicit line in Context: local run state "is not sent to the LLM", which sees only the conversation [14]. smolagents frames agency itself as a matter of Control flow, from a router to a multi-step loop to multiple agents, and exposes the main Output decision as a choice of class: `CodeAgent` writes actions as Python, `ToolCallingAgent` as JSON tool calls, following Wang et al. [24] [6]. AutoGPT, an early autonomous agent from 2023, already had all five: a goal as input, history plus retrieved vector memory as context, a JSON response of thoughts and a command, named commands as output, and an unbounded loop gated by human authorization [21]. Its safety model, a person approving each command unless a "continuous mode" was enabled, is the same decision that current harnesses express as approval modes.

6.3 The differences are implementation decisions

Reading the tables by row rather than by column shows each difference as a choice along an axis within one primitive.

Table 3: Command-line coding agents, by primitive.

	Claude Code	Codex CLI	Gemini CLI	Aider	Cline
Input	Interactive terminal; `-p` with piped stdin or a JSON stream; messages queued mid-turn; background output as events	Interactive terminal; `codex exec` for scripts and CI, emitting JSONL events; SDK	Interactive terminal with `@file` and `!` shell; headless JSON output; ACP mode for editors	Terminal chat with `code`, `ask`, `architect` and `help` modes	Editor side panel; terminal output and linter or compiler errors
Context	System prompt, tools, layered `CLAUDE.md` (or `AGENTS.md`), auto-memory, skill descriptions; clears old tool outputs then summarizes; deferred tool schemas loaded on demand	Layered `AGENTS.md` from home to working directory, capped at 32 KiB; compaction into a hand-off summary at a token limit	Layered `GEMINI.md`; compression at half the window keeps the newest 30% and summarizes the rest	Repository map of signatures from tree-sitter, ranked by PageRank over a dependency graph, within a token budget; chat history summarized	Project structure; automatic summarization near the limit, with rule-based truncation as fallback
Model interface	Claude models via the Anthropic API or cloud providers; fallback model chain; streaming	OpenAI Responses API, streamed; other providers through configurable model providers	Gemini models via the Google GenAI SDK, streamed; optional local routing	Many providers through LiteLLM; edits parsed from text, not tool calls	Many providers; native tool-call schemas
Output	About twenty tools: shell, read, exact-string edit, write, search, web, sub-agent, plan mode, notebook, language server; MCP tools	Shell with resumable sessions, grammar-constrained `apply_patch`, plan updates, image view, web search, multi-agent tools; MCP tools	Shell, file read and write, exact-string replace, glob and grep, web search and fetch, to-do and plan tools; MCP tools	Edit formats: whole file, search/replace, unified diff; architect mode pairs a planning model with an editing model	About twenty tools: command, read, write, replace, patch, search, browser, web, MCP, sub-agents, completion
Control flow	Loop until a reply has no tool calls; optional turn and budget limits; sub-agents with their own context; plan mode	Loop over the Responses API; sub-agents on by default; plan mode; persistent goal	Loop with a hard cap of 100 turns and loop detection; sub-agents exposed as tools; plan mode	One edit per request, then lint and test, retrying failures up to three times	Plan and Act modes, each with its own model; ends on `attempt_completion`; sub-agents

Table 4: Research systems for software engineering, by primitive.

	SWE-agent	mini-SWE-agent	OpenHands	Agentless
Input	An issue or task statement, with optional demonstrations	A task string rendered into templates	User messages and environment observations in one event stream; issue triggers	An issue and its repository
Context	Templates for each step; history processors that keep the last five observations and collapse older ones	Linear history: every step appends, nothing is processed	Chronological event history; a summarizing condenser keeps the first events and a recent tail and summarizes the middle; `AGENTS.md` and skills loaded by trigger	Hierarchical localization: repository tree, then file skeletons, then edit locations; embedding retrieval in parallel
Model interface	LiteLLM; pluggable parsers for thought-and-action text or native tool calls	LiteLLM and other gateways; no tool-call interface, so any model works	LiteLLM; tool calls parsed from text for models without native support	Plain completions; no tool calls
Output	An agent-computer interface: file viewer of 100 lines, search with capped results, an edit command that rejects edits failing a linter	Bash only, each command in a fresh subprocess	Code actions: Python and shell in a sandbox; a browser; editing utilities	Search/replace patches; forty candidate patches per issue
Control flow	Open loop ending on `submit`; retries after malformed output; a cost budget that auto-submits; a meta-agent that picks the best of several attempts	A `while` loop ending on a completion marker; step, cost and time limits	A step loop; delegation to specialized agents; stuck detection	A fixed pipeline: localize, repair, validate; a majority vote over normalized patches

Table 5: Agent frameworks, by primitive, with AutoGPT (2023) for historical comparison.

	LangGraph	OpenAI Agents SDK	smolagents	AutoGPT (2023)
Input	Graph invocation with an initial state; resumption values after an interrupt	Run input to a runner; session history; approvals to resume a paused run	A task string; outputs of managed agents	A goal set by the user; per-command authorization
Context	A user-defined state schema, merged by per-key reducers; a message list with an ID-aware reducer; a long-term store	Instructions, input and tool results only; local run context is explicitly *not* sent to the model; handoff input filters	Memory written to messages each step; managed agents described in the manager's system prompt; optional planning steps	Recent history plus vector-memory retrieval, fitted to the token limit
Model interface	Left to user code inside nodes	Responses or Chat Completions models; custom providers; beta LiteLLM adapter	Many model classes, including local inference and LiteLLM	OpenAI chat API; JSON responses with thoughts and a command
Output	Left to user code inside nodes; tool nodes	Hosted tools, local shell and patch tools, function tools from signatures, agents as tools; approval before chosen tools	Code actions (Python) or JSON tool calls; sandboxed executors	Named commands with arguments (files, web, memory)
Control flow	An explicit graph: nodes, conditional edges, commands that route or hand off to a parent graph, fan-out; super-step limit	A runner loop: model call, then final output, handoff, or tool calls; handoffs as tools; turn limit	A multi-step loop ending on `final_answer` or a step limit; answer checks; managed agents	An infinite loop with human authorization per command; a continuous mode with a limit

INPUT**: where requests come from.** All of the coding agents accept both an interactive person and a non-interactive caller (a command-line flag, a JSON event stream or an SDK), so the same harness serves a developer and a continuous-integration job. Research systems take a task statement, usually an issue. These are different sources feeding the same primitive.

CONTEXT**: what the model is shown.** Three designs recur. The first is a *project instruction file* merged from several directory levels: `CLAUDE.md`, `AGENTS.md` and `GEMINI.md` are the same decision under three names, and Claude Code and OpenHands read `AGENTS.md` as well. The second is a strategy for *keeping history within the window*, and here the axis is clearest: mini-SWE-agent processes nothing; SWE-agent elides old observations, which its authors found better than keeping full history [28]; Claude Code, Codex CLI, Gemini CLI, OpenHands and Cline summarize, differing in when they trigger and what they keep verbatim. The third is *what of the project to show*, from nothing (the agent explores through its tools) to Aider's ranked repository map to Agentless's staged localization. All are answers to one question: what should the model be shown on this call?

MODEL INTERFACE**: one provider or many, native or parsed.** Model developers' harnesses bind to their own API, while research and community harnesses route through a translation layer, most often LiteLLM, to reach many providers. A second axis is whether actions arrive as native tool calls or are parsed from text; mini-SWE-agent, Aider and Agentless parse text, and SWE-agent and OpenHands support both. Both axes trade portability against the use of provider-specific features.

OUTPUT**: how many tools, and in what format.** The axis runs from a single shell (mini-SWE-agent), through a small purpose-built interface (SWE-agent), to rich sets of around twenty tools (Claude Code, Cline, Gemini CLI) and to code as the action format (OpenHands, following Wang et al. [24]). Edit formats are a sub-axis: exact string replacement (Claude Code, Gemini CLI, Cline), a grammar-constrained patch language (Codex CLI), and search/replace or unified diffs parsed from text (Aider, Agentless). Each harness also extends its tool set through the Model Context Protocol, which is a decision about how OUTPUT discovers tools, not a new primitive.

CONTROL FLOW**: who decides what happens next.** At one end, Agentless fixes the sequence of model calls in code; at the other, Claude Code, Codex CLI and mini-SWE-agent run until the model stops requesting actions. Between them lie explicit completion actions (`submit`, `attempt_completion`), retry loops driven by tests (Aider), best-of-several attempts (SWE-agent's meta-agent), delegation to sub-agents with separate contexts, and modes that change what is allowed (plan modes, Aider's architect mode). Stopping rules vary in the same way: a hard turn cap (Gemini CLI), optional turn and budget limits (Claude Code's SDK), and cost or time limits (mini-SWE-agent, SWE-agent).

6.4 Production concerns

Table 6 shows the production concerns of the coding agents. The same pattern holds: each concern is answered inside a primitive, and the harnesses differ in their answers rather than in which concerns they have.

Two designs for safety stand out. *Isolation* contains the damage an action can do: the model developers' harnesses use operating-system sandboxes, and the research systems run in containers. *Reversibility* makes damage undoable: Aider commits every change, Cline and Gemini CLI keep a shadow git repository, and Claude Code checkpoints files. Most harnesses combine the two with

Table 6: Production concerns in command-line coding agents.

	Claude Code	Codex CLI	Gemini CLI	Aider	Cline
Persistence	Session transcripts, resume and fork; file checkpoints before each prompt with rewind	Session files, resume and fork; no built-in rewind (git recommended)	Auto-saved sessions, resume; checkpoints in a shadow git repository; rewind	Every change auto-committed to git; undo	Checkpoints in a shadow git repository after each tool use; restore files, task, or both
Safety & security	Permission modes; deny/ask/allow rules; OS sandbox (Seatbelt, bubblewrap) with a network allow-list	Sandbox modes from read-only to full access, network off by default; approval policies; OS sandbox (Seatbelt, bubblewrap with seccomp)	Approval modes; a rule-based policy engine; sandboxes from Seatbelt to containers and gVisor; trusted folders	No sandbox; edits the working tree under version control	Approval for every edit and command, with per-category auto-approval; no sandbox
Observability	Hooks at about thirty events; OpenTelemetry	Hooks; OpenTelemetry	Hooks around agent, model and tool calls; OpenTelemetry	Commit history	Checkpoint history
Resilience	Configurable retries and timeouts; fallback models	Request and stream retries; idle timeout	Retries with backoff; model fallback	—	—
Performance	Prompt caching; read-only tools run concurrently	Prompt cache key; parallel tool calls	Token caching; parallel tool calls batched	Token-budgeted repository map	Context summarization

approvals, which keep a person in the loop. These are different answers to the safety question, located in Output and Control flow.

6.5 Two limit cases

A harness without a loop. Agentless is presented as an alternative to agents: a fixed pipeline of localization, repair and validation in which "the LLM [does not] decide future actions" [27]. In our terms it is still a harness with all five primitives; its Control flow is a fixed sequence written in code rather than an open loop, and its Context does the heavy lifting through staged localization. Its competitive results at the time of publication show that the axis within Control flow is a real design choice, not a ladder with agents at the top.

A harness with almost nothing. mini-SWE-agent describes itself as a hundred-line agent: one tool (bash), a linear history with no processing, and no tool-call interface. Its authors report that it resolves over 74% of SWE-bench Verified with a strong model [23]. It shows the primitives at their smallest: each is present, and each is a single decision. That a harness this small is competitive, given a capable model, supports the position that the primitives, not any particular elaboration of them, are what make an agent.

6.6 Convergence

The three harnesses built by model developers, written independently, converged on nearly the same decisions: a layered instruction file in Context, summarization to keep history in the window, a rich tool set extensible through MCP in Output, sub-agents and plan modes in Control flow, and, across the production concerns, operating-system sandboxes, rule-based permissions, hooks and OpenTelemetry. We read this convergence as evidence that the primitives pose the right questions: independent teams, answering the same five questions under similar constraints, arrive at similar answers.

7 Implications

Report the harness. If *Agent* = *Harness*(*Model*), then a benchmark score belongs to the pair, not to the model. The evidence is consistent. Holding GPT-4 Turbo fixed, Yang et al. [28] report 1.31% of the full SWE-bench [7] resolved with retrieval-augmented generation and 12.47% with their agent; on SWE-bench Lite, a shell-only agent resolved 11.00% and SWE-agent 18.00%, with only the harness changing. Anthropic defines an agent as "the combination of an AI model and the software scaffolding around it" and notes that performance "can vary significantly based on this scaffolding, even when using the same underlying AI model" [1]. METR found that tailoring its scaffold to one model more than doubled the length of tasks it could complete [11], and the Holistic Agent Leaderboard evaluates models, scaffolds and benchmarks as separate dimensions [9]. The effect is not always large: METR later found that neither Claude Code nor Codex outperformed its default scaffolds for the models they were built for [12], which suggests that harness effects are greatest where the primitives differ most, and shrink between mature harnesses that have converged (§6). Either way, a score without its harness is underspecified, a point Kapoor et al. [8] make about agent evaluation more broadly. We suggest that agent results report the harness alongside the model, described primitive by primitive: the tools offered and their output formats (Output), how context is managed (Context), the stopping rules and any fixed pipeline (Control flow), and the decoding settings (Model interface). Comparisons between models are then fair only within one harness, and comparisons between harnesses only with one model.

Locate failures. When an agent fails, the primitives give a short list of places to look. The model may never have been shown what it needed (Context); it may have asked for the right action and the action may not have happened as intended (Output); the loop may have stopped too early or run too long (Control flow); the response may have been misread (Model interface). Only when all of these are ruled out does the failure belong to the model. In our own work, building the harness around a capable model first, until it passed every task in our evaluation, meant that every failure that remained when a smaller model was swapped in could be attributed to the model.

Port by changing one primitive. Because only Model interface knows which model is inside, a harness can be moved to a new model or provider by changing that primitive alone, translating requests and responses to the new format. Harnesses that let model-specific details leak into other primitives (assuming a provider's tool-call format in Output, or a provider's error messages in Context management) lose this property.

Train the model in its harness. A model post-trained for many harnesses must guess which one it is in; a model trained inside one harness learns its tool format, the shape of its results and its stopping rule. The primitives make explicit what such training must match: the context layout the model will see (Context), the format in which it must request actions and the results it will read (Output), and the loop it will run in (Control flow). In our experiments, a small model instruction-tuned on verified sessions recorded in its own harness rose from 2 to 18 of 32 held-out tasks in that harness, while the same base model post-trained by its developers for general use scored 14.

8 Alternative Views

Memory is a sixth primitive. Memory is prominent in agent architectures [15, 22], and could be argued to deserve its own place. We place it instead as a store in the world that CONTEXT reads and OUTPUT writes, governed by the persistence concern. The model has no memory of its own, and every implementation we examined (memory files loaded into the instructions, stores searched through tools, summaries carried forward) is a decision about what CONTEXT includes and which actions OUTPUT offers. Making memory a primitive would split one decision, what the model is shown, across two components.

Planning is a sixth primitive. Planning is either something the model does in its own tokens, which needs no harness support, or a structure the harness imposes: a planning mode that restricts actions, a plan kept in a file, a separate planning call. Each of these is CONTROL FLOW (which steps run, in what order, under what restrictions) or CONTEXT (where the plan is kept and shown).

Multi-agent systems are something else. A system of several agents, each a model in a harness, is itself a harness whose OUTPUT includes delegating to another agent and whose INPUT includes that agent's result. Sub-agents, hand-offs between agents and orchestrator–worker patterns are CONTROL FLOW decisions at this outer level. The primitives compose: a harness of harnesses still has five.

The harness will be absorbed into the model. Models increasingly learn behaviours that harnesses once imposed, such as deciding when to stop, calling tools in a trained format, and reasoning before acting. But a model cannot open a file, keep a log, or refuse a command on its own: whatever is learned, something must still carry tokens to the world and back. Training moves *policy* into the model (what to do); the *mechanism* (how anything happens) remains in the harness. If anything, the primitives become more important as models are trained inside a particular harness (§7), because the harness then defines the world the model was trained for.

The primitives are too coarse to be useful. Five boxes can describe anything, and so might explain nothing. Our answer is that the primitives are a coordinate system, not an explanation: their value is in locating a design decision, a failure or a difference between two systems in one place, so that it can be studied, compared and changed independently of the rest. The design spaces in §4 and the case study in §6 are where the content lies.

9 Conclusion

We have argued that every LLM agent is a model wrapped in a harness, and that every harness is built from five primitives: INPUT, CONTEXT, MODEL INTERFACE, OUTPUT and CONTROL FLOW. The production concerns that make a harness safe and dependable are questions asked of these primitives, not additional parts. The harnesses in wide use today, however different they appear, make different decisions within the same five primitives.

The harness is where most of an agent's behaviour is decided: what it knows about itself and its situation, what it is able and allowed to do, how it recovers from failure, and when it goes again. It deserves to be named, decomposed and reported with the same care as the model inside it. We hope the vocabulary proposed here helps make that routine.

References

[1] Anthropic. Raising the bar on SWE-bench verified with Claude 3.5 Sonnet. https://www.anthropic.com/research/swe-bench-sonnet, 2025.

[2] Anthropic. Claude Code documentation. https://code.claude.com/docs/en/how-claude-code-works, 2026. Accessed October 2026.

[3] Cline. Cline. https://github.com/cline/cline, 2025.

[4] Paul Gauthier. Aider: AI pair programming in your terminal. https://aider.chat, 2024.

[5] Google. Gemini CLI. https://github.com/google-gemini/gemini-cli, 2026. Commit 42c5182; accessed October 2026.

[6] Hugging Face. smolagents documentation. https://huggingface.co/docs/smolagents, 2026. Accessed October 2026.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[8] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024.

[9] Sayash Kapoor et al. Holistic agent leaderboard, 2025.

[10] LangChain. LangGraph documentation. https://docs.langchain.com/oss/python/langgraph/overview, 2026. Accessed October 2026.

[11] METR. An update on our preliminary evaluations of Claude 3.5 Sonnet and o1. https://metr.org/blog/2025-01-31-update-sonnet-o1-evals/, 2025.

[12] METR. Measuring time horizon using Claude Code and Codex. https://evals.alignment.org/notes/2026-02-13-measuring-time-horizon-using-claude-code-and-codex/, 2026.

[13] OpenAI. Codex CLI. https://github.com/openai/codex, 2026. Commit 2f05c0a; accessed October 2026.

[14] OpenAI. OpenAI Agents SDK documentation. https://openai.github.io/openai-agents-python/, 2026. Accessed October 2026.

[15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[16] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

[17] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2302.04761.

[18] Erik Schluntz and Barry Zhang. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024.

[19] Thariq Shihipar. Building agents with the Claude Agent SDK. https://claude.com/blog/building-agents-with-the-claude-agent-sdk, 2025.

[20] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2303.11366.

[21] Significant Gravitas. Auto-GPT: An autonomous GPT-4 experiment. https://github.com/Significant-Gravitas/AutoGPT/tree/v0.2.0, 2023. Version 0.2.0.

[22] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. arXiv:2309.02427.

[23] SWE-agent team. mini-SWE-agent. https://github.com/SWE-agent/mini-swe-agent, 2025.

[24] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024. arXiv:2402.01030.

[25] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025. arXiv:2407.16741.

[26] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and Graham Neubig. The OpenHands software agent SDK: A composable and extensible foundation for production agents, 2025.

[27] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024. Published at FSE 2025 as "Demystifying LLM-based Software Engineering Agents".

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. arXiv:2405.15793.

[29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.

[30] J. Young. Effective harnesses for long-running agents. https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents, 2025.

A A Reference Harness in Fifty Lines

The listing below is *quark*, the reference harness used to argue for each primitive's necessity in §4. It is the complete file `harness/quark.py` from the accompanying repository, without the dependency header. Each function is one primitive. It gives the model a single tool, a shell, and runs commands on the host without asking; it is a teaching artifact, and the production concerns of §5 are what it lacks.

```
import subprocess, os, datetime
from anthropic import Anthropic

model = Anthropic()
bash = {"name": "bash", "description": "Run a shell command",
        "input_schema": {"type": "object", "properties": {"command": {"type": "string"}}, "required": ["
     command"]}}


def capture_input():
    return input("\n> ")


def assemble_context(conversation):
    instructions = f"You are quark, an agent. You act through bash, in {os.getcwd()}. Today is {datetime.date.
     today()}."
    return {"system": instructions, "tools": [bash], "messages": conversation}


def request_response(context):
    return model.messages.create(model="claude-sonnet-5-5", max_tokens=16384, **context)


def handle_output(response):
    tool_results = []
    for block in response.content:
        if block.type == "text":
            print("< " + block.text.replace("\n", "\n  "))
        if block.type == "tool_use":
            print(f"$ {block.input['command']}")
            ran = subprocess.run(block.input["command"], shell=True, capture_output=True, text=True)
            tool_results.append({"type": "tool_result", "tool_use_id": block.id, "content": ran.stdout + ran.
     stderr or "(no output)"})
    return tool_results


def control_flow():
    conversation = []
    while True:
        conversation.append({"role": "user", "content": capture_input()})
        while True:
            response = request_response(assemble_context(conversation))
            conversation.append({"role": "assistant", "content": response.content})
            tool_results = handle_output(response)
            if not tool_results:
                break                                   # done: hand back to the person
            conversation.append({"role": "user", "content": tool_results})   # a tool ran: go again


control_flow()
```